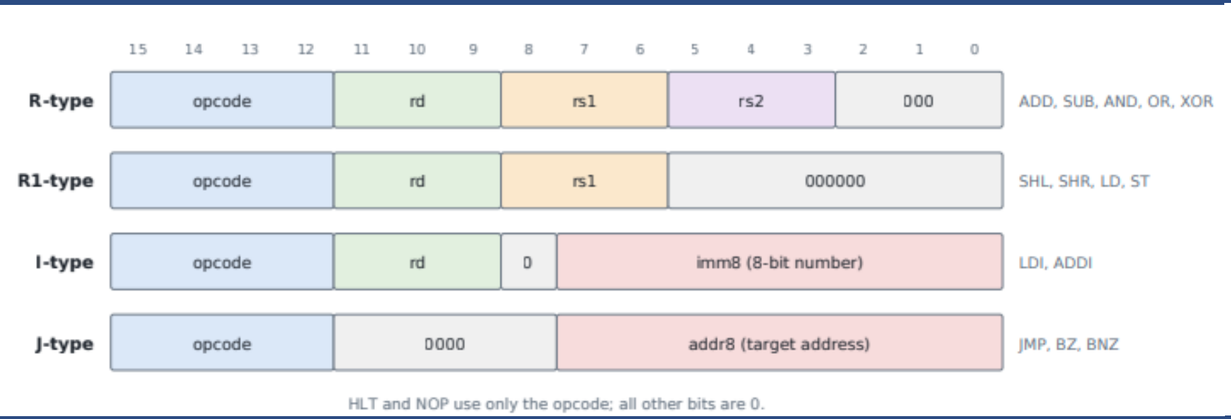


Name	NGUYEN DINH DUNG
Roll number	HE201882
Branch / Section / Batch	FPT UNIVERSITY HANOI
Project number assigned (1–10)	PROJECT 2
Project title	8-BIT RISC PROCESSOR - 5 PIPELINE
Vivado version used	XILINX VIVADO 2026.1
Date submitted	23/10/2026

Specification of the processor

Parameter	Value	Meaning
Data width	8 bits	Registers, ALU and data memory work on numbers 0–255
Instruction width	16 bits, fixed	Every instruction is exactly one 16-bit word
Registers	8 × 8-bit (R0–R7)	Fast storage inside the CPU; R0 is an ordinary register
Program counter	8 bits	Can address 256 instructions
Instruction memory	256 × 16-bit ROM	Holds the program
Data memory	256 × 8-bit RAM	Read by LD, written by ST
Architecture	Harvard	Separate instruction and data memories
Pipeline	5 stages: IF, ID, EX, MEM, WB	Up to 5 instructions in progress at once
Flags	Z (zero), C (carry / borrow)	Set by arithmetic and logic instructions
Instructions	16 (4-bit opcode)	Arithmetic, logic, shift, load, store, jump, branch, halt
Hazard handling	Forwarding, bypass, 1-cycle load-use stall, 2-cycle branch flush	Hardware keeps results correct
Target	Xilinx FPGA through Vivado	Fully synthesizable Verilog-2001

Instruction set architecture (ISA)



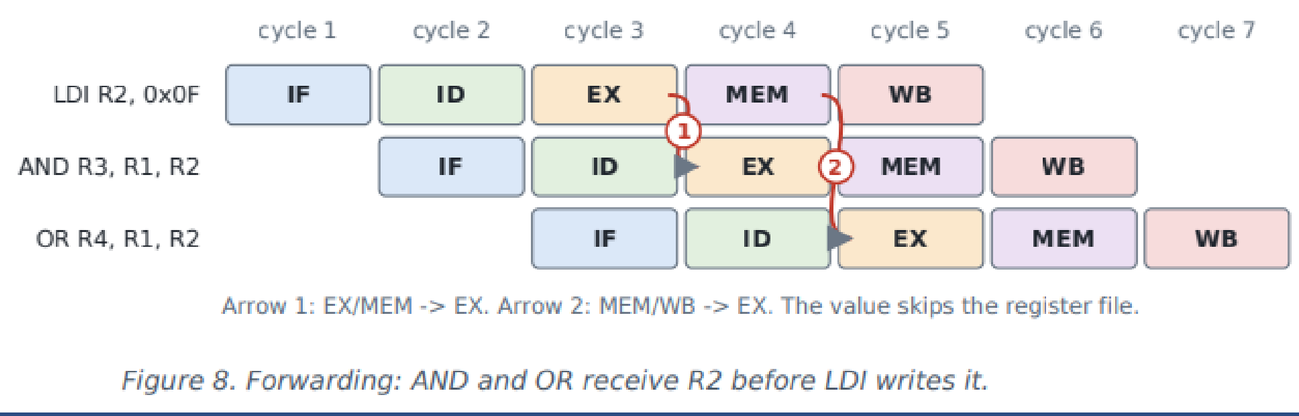
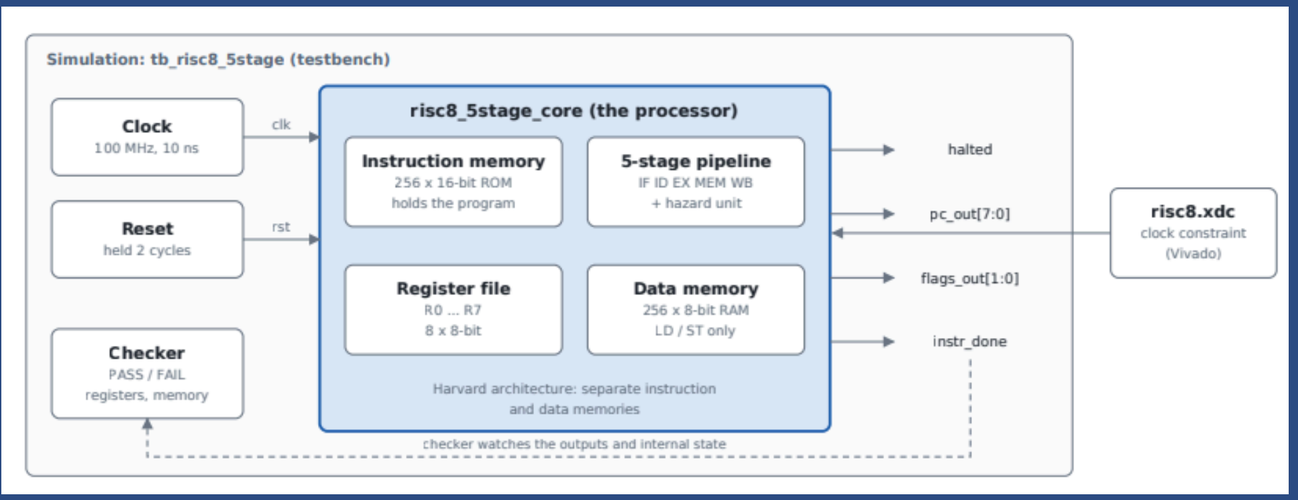
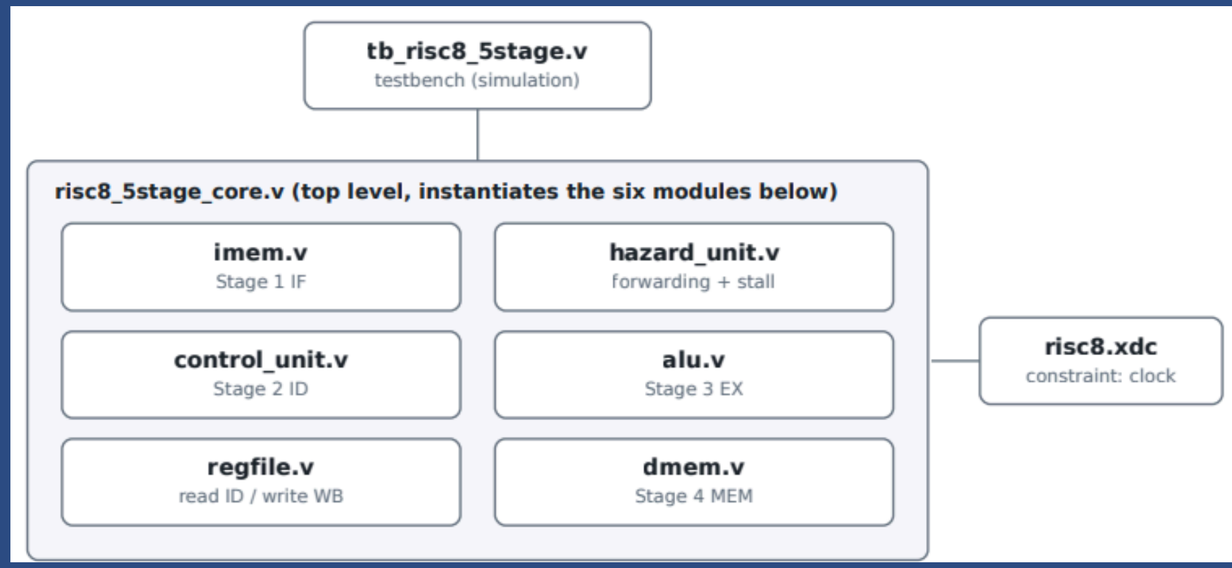


Figure 8. Forwarding: AND and OR receive R2 before LDI writes it.

- Main Figure : Illustrates the 16-bit bit layout (from bit 15 down to 0) across the four instruction formats (R-type, R1-type, I-type, and J-type), highlighting the consistent placement of the opcode, destination register (rd), source registers (rs1, rs2), immediate value (imm8), and target address (addr8).
- Instruction formats: Fixed 16-bit instructions with the top 4 bits serving as the opcode, split into four formats (R-type, R1-type, I-type, J-type) to allow the decoder to read key fields immediately.
- The 16 instructions: Consists of 16 instructions grouped into NOP/HLT, arithmetic/logic (ADD, SUB, AND, OR, XOR, SHL, SHR, ADDI), memory load/store (LDI, LD, ST), and jumps/branches (JMP, BZ, BNZ).
- Flags: Features the Z (zero) flag for zero results (used for loops and if statements) and the C (carry) flag for overflows, borrows, or shifted-out bits. -
- Addressing modes: Supports four modes including Register, Immediate, Register indirect, and Absolute (branch). -
- Encoding: Translates assembly instructions into their corresponding binary and hexadecimal machine code.

Top-level block diagram and file structure





- System View (Figure 2): The processor core (risc8_5stage_core) takes only two inputs (clk and rst) and provides four observation outputs (halted, pc_out, flags_out, and instr_done) which can drive LEDs on an FPGA board.
- File Hierarchy (Figure 3): The testbench (tb_risc8_5stage.v) instantiates the top-level core (risc8_5stage_core.v), which in turn wires together six main modules along with a clock constraint file (risc8.xdc).
- Module Summary Table: Lists the core files and their primary roles, including instruction memory (imem.v), control unit (control_unit.v), register file (regfile.v), hazard unit (hazard_unit.v), arithmetic logic unit (alu.v), and data memory (dmem.v).

Pipelining theory

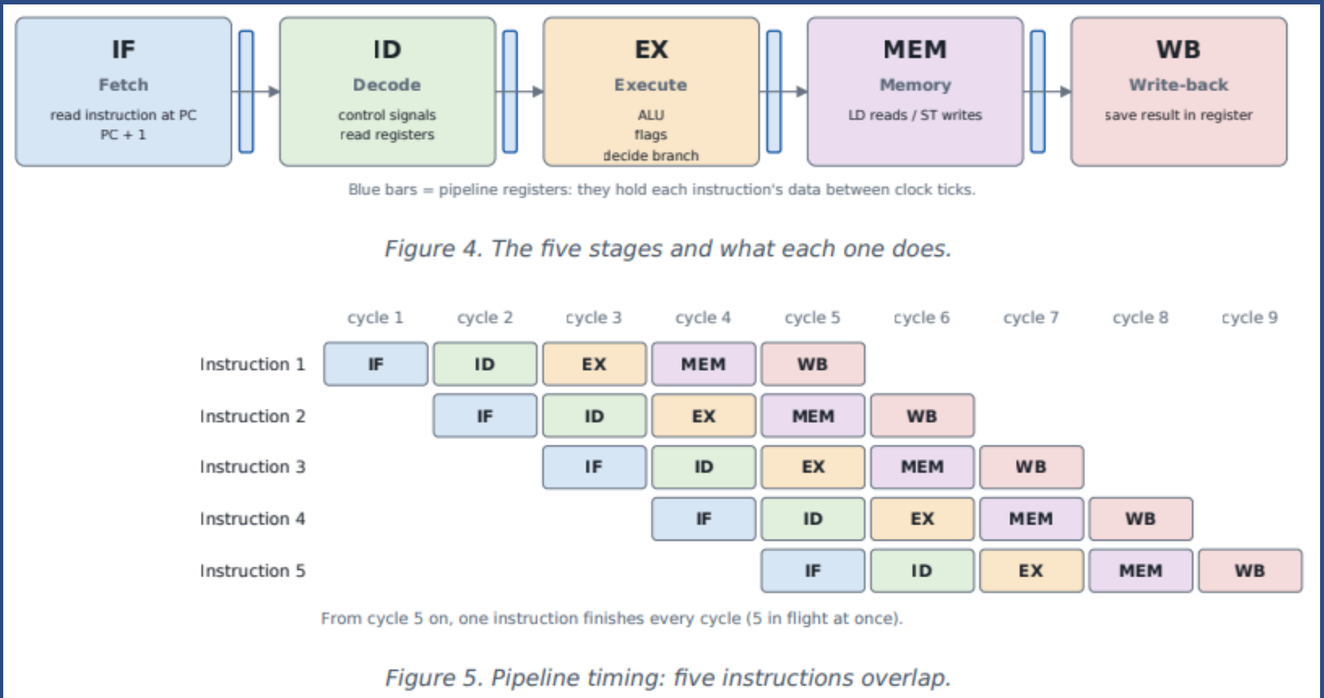
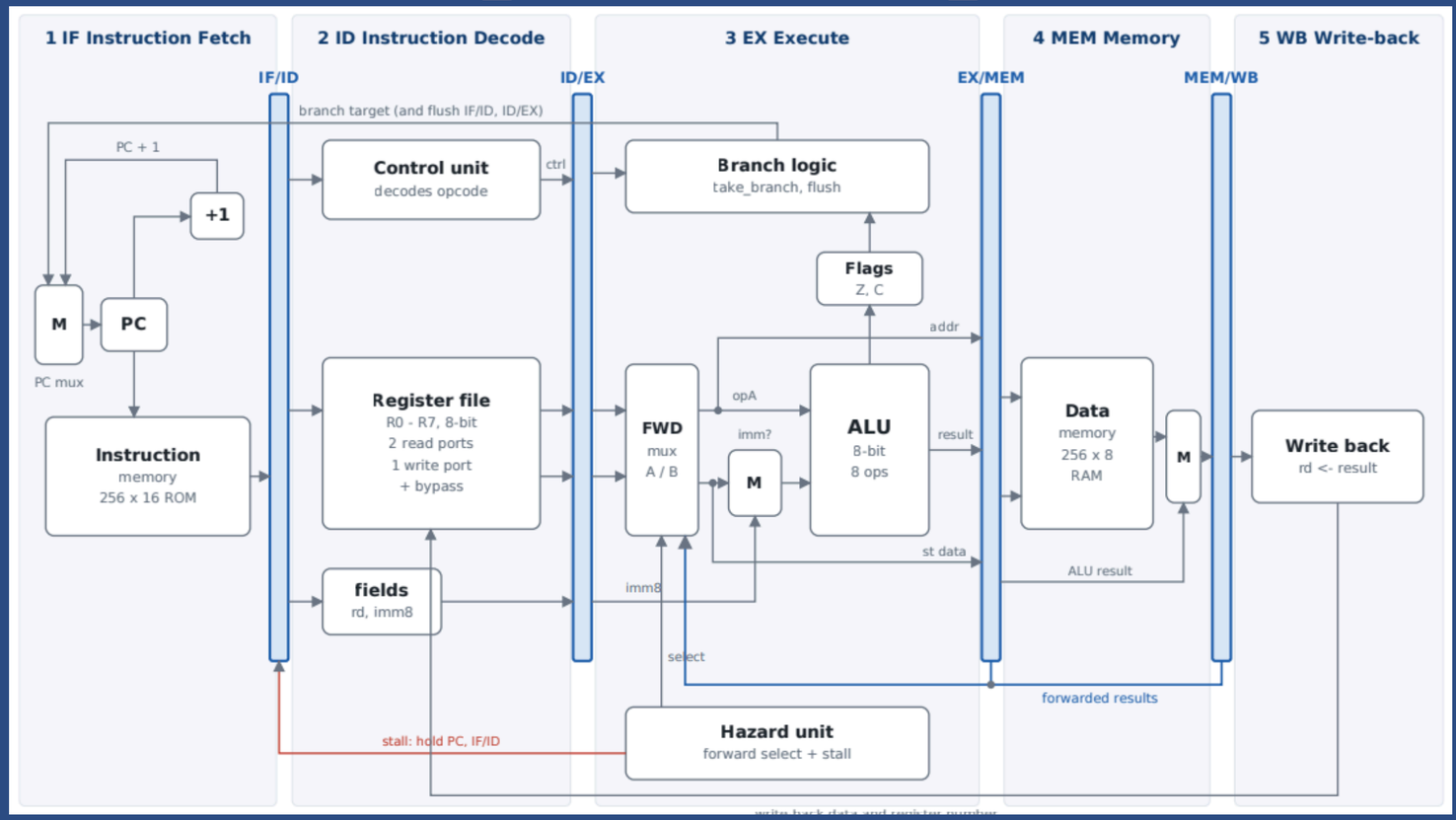


Figure 4. The five stages and what each one does.

Figure 5. Pipeline timing: five instructions overlap.

- (The five stages and what each one does): Visually illustrates the 5 pipeline stages (IF, ID, EX, MEM, WB) along with the pipeline registers (represented by blue bars) placed between stages to hold instruction data between clock ticks.
- (Pipeline timing: five instructions overlap): A timing diagram showing how five instructions overlap across clock cycles from cycle 1 to cycle 9, demonstrating that starting from cycle 5, one instruction finishes every single cycle.

The complete datapath



- Complete Datapath Architecture: The slide presents the full hardware datapath integrating all 5 pipeline stages (IF, ID, EX, MEM, and WB).
- Pipeline Registers: Blue vertical bars (IF/ID, ID/EX, EX/MEM, MEM/WB) hold instruction data and control signals between clock cycles.
- Core Components: Features the instruction memory, control unit, register file (R0–R7), ALU, data memory (256 x 8 RAM), and branch logic.
- Hazard Handling: The bottom Hazard unit manages data forwarding (forwarded results) and pipeline stalls (stall: hold PC, IF/ID) to ensure correct execution.

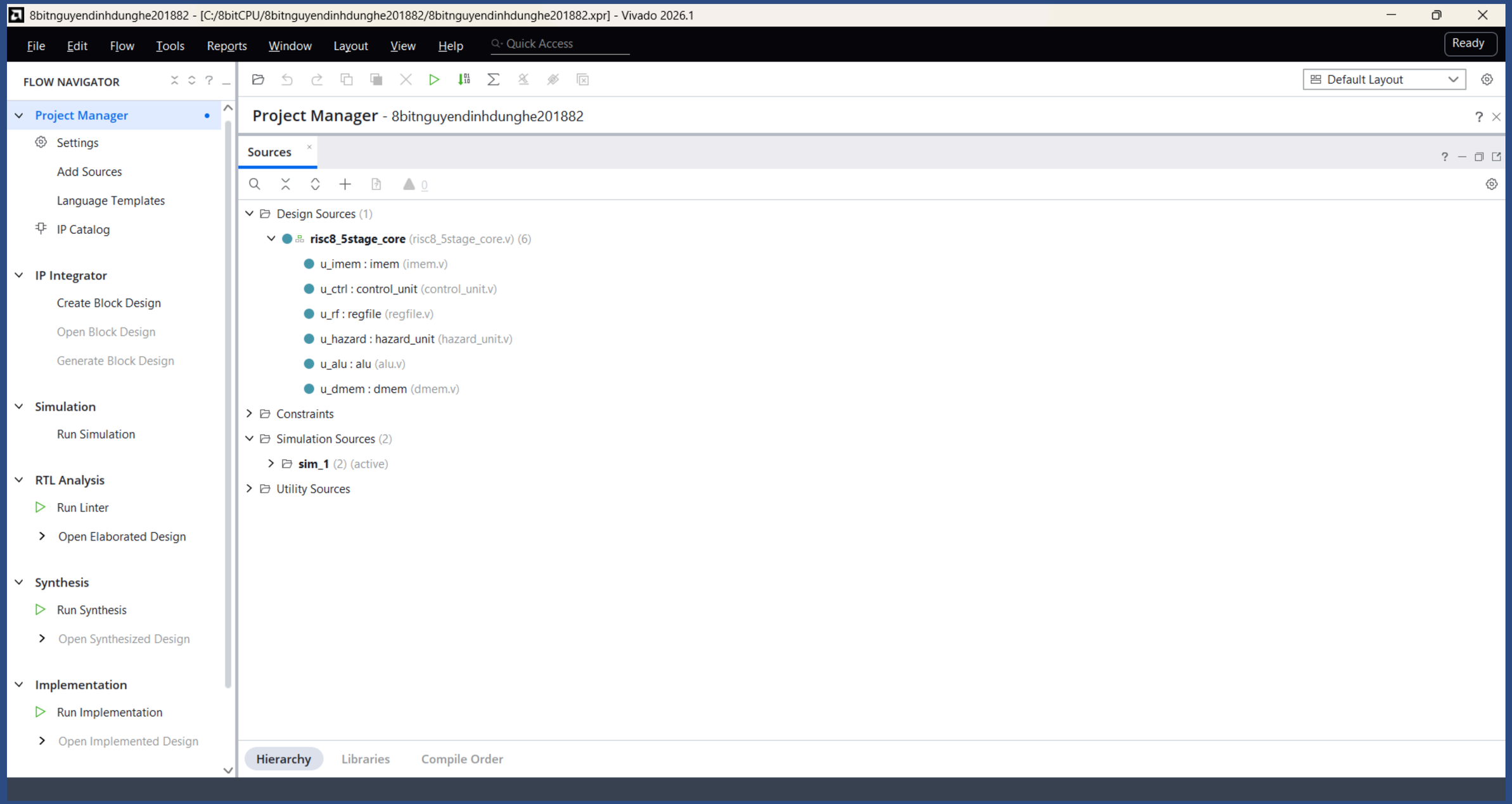
Instruction walk-throughs

Stage	ADD R3, R1, R2	LD R5, [R4]	ST R2, [R4]	BZ 0x0A
IF	fetch 0x3650	fetch 0xAB00	fetch 0xB500	fetch 0xD00A
ID	read R1, R2; reg_we=1, flag_we=1, alu_op=ADD	read R4; reg_we=1, mem_to_reg=1	read R4 (A) and R2 (B); mem_we=1	br_z=1; target 0x0A
EX	ALU: R1 + R2; set Z, C	address = R4 passed on	address = R4, data = R2 passed on	if Z = 1: PC = 0x0A, flush 2
MEM	pass result	read memory[R4]	write memory[R4] = R2	–
WB	R3 = result	R5 = loaded byte	nothing	nothing

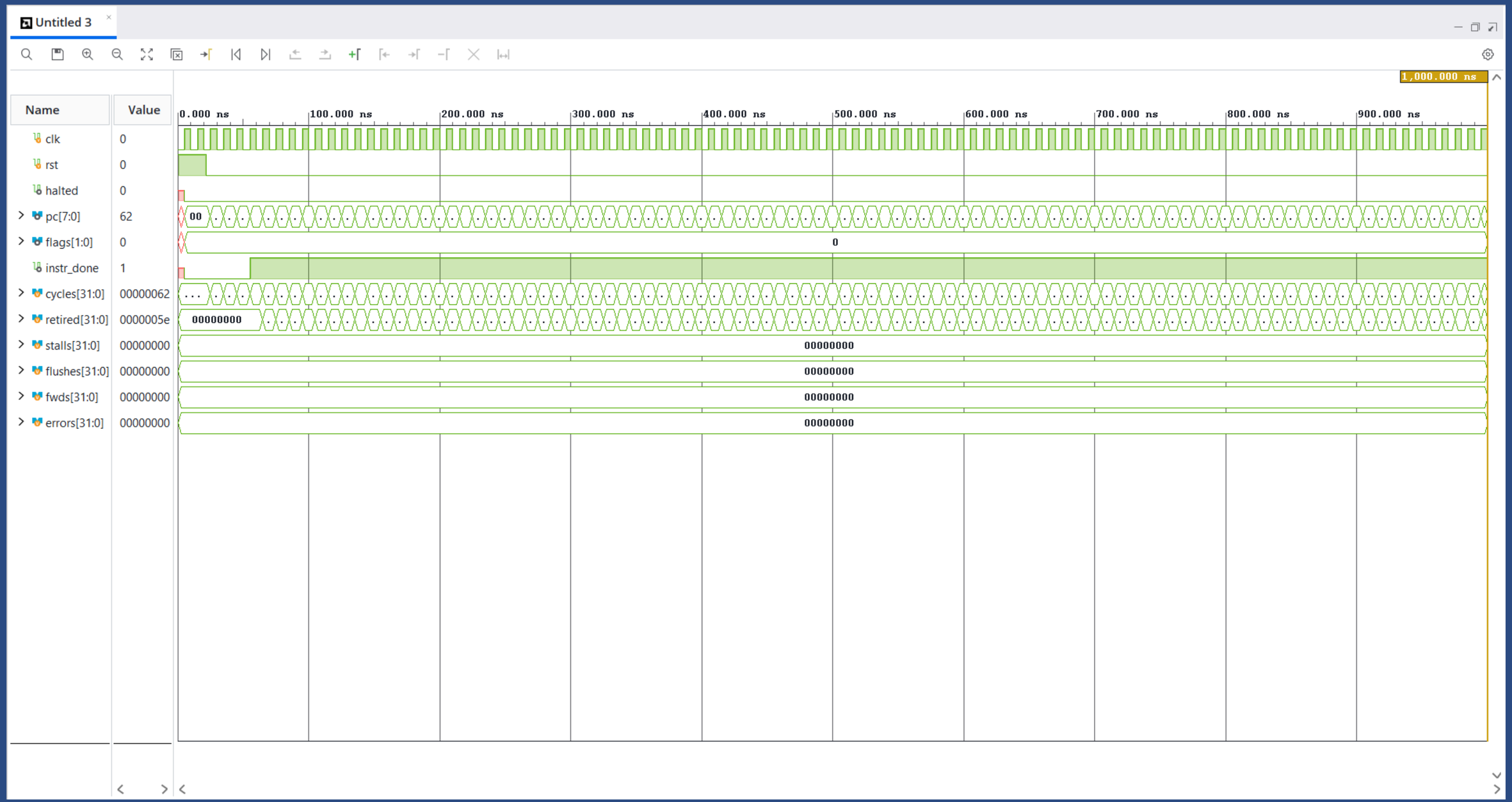
Design decisions and trade-offs

Decision	Why	Alternative
Branches resolved in EX	The Z flag is produced by the ALU in EX, so the decision is always based on up-to-date flags	Decide in ID with extra compare logic: 1-cycle penalty, more hardware
Fixed 16-bit instructions	8 bits are too few for opcode plus registers; fixed length keeps decoding simple	Variable length (CISC): harder to pipeline
Asynchronous memory read	Instruction and data are available in the same cycle as the address	Synchronous block RAM: one more cycle of latency
Write-through register bypass	Removes the distance-3 hazard with a tiny multiplexer	Write in first half of cycle, read in second half
Forwarding gated by uses_a / uses_b	No false forwards or stalls from unused fields	Simpler logic, but occasional needless stalls
Program built into imem.v	No external file; Vivado cannot lose it	$readmemh from a .hex file
Debug outputs (pc_out, flags_out, instr_done)	Observable on LEDs; no unused-signal warnings	Internal signals only

code sources



wave form

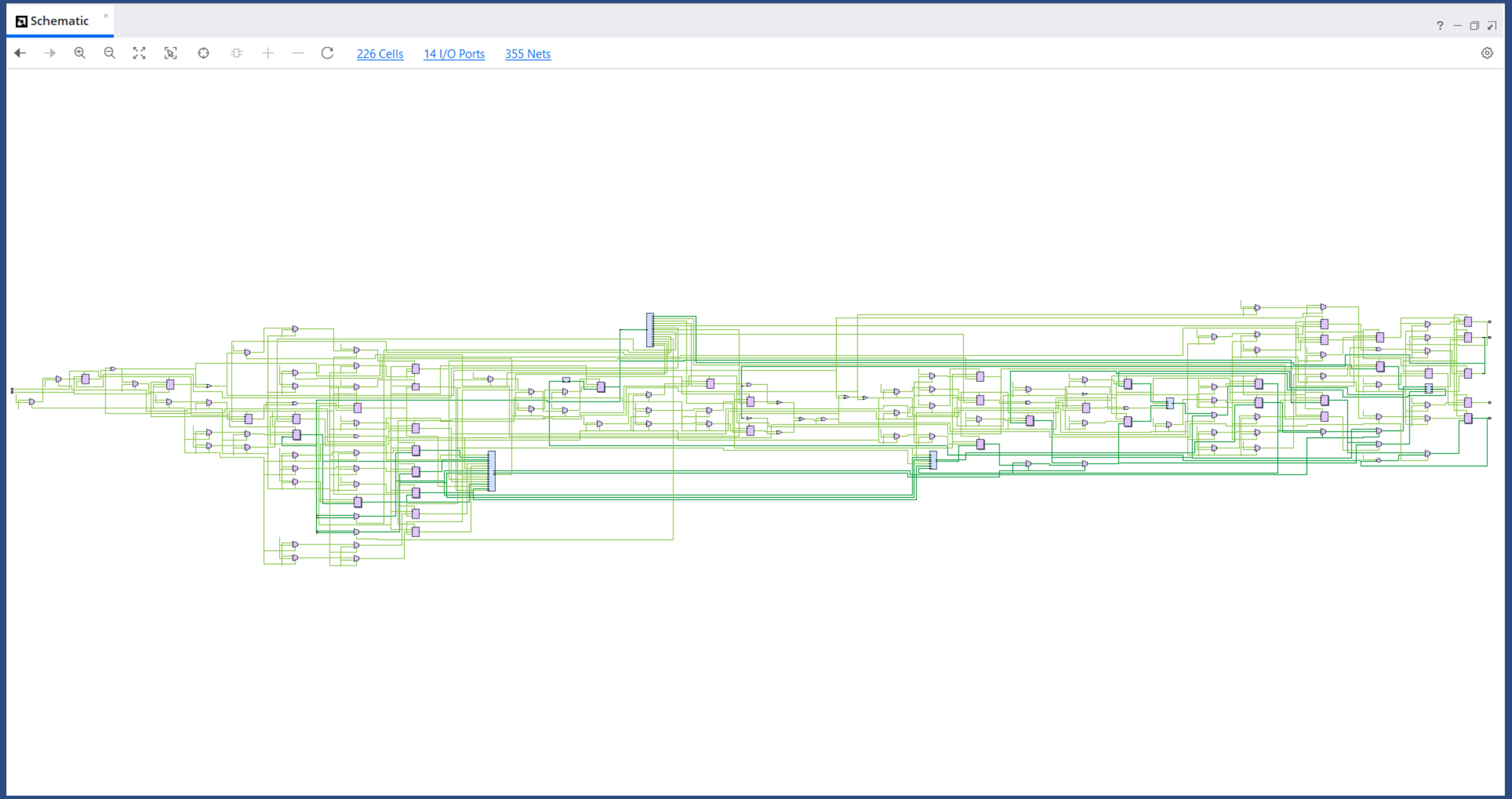

Untitled 3
Name
Value
clk
0
rst
0
halted
0
pc[7:0]
62
flags[1:0]
0
instr_done
1
cycles[31:0]
00000062
retired[31:0]
0000005e
stalls[31:0]
00000000
flushes[31:0]
00000000
fwds[31:0]
00000000
errors[31:0]
00000000
0.000 ns
100.000 ns
200.000 ns
300.000 ns
400.000 ns
500.000 ns
600.000 ns
700.000 ns
800.000 ns
900.000 ns
1,000.000 ns
00
0
00000000
00000000
00000000
00000000
00000000

tcl console check

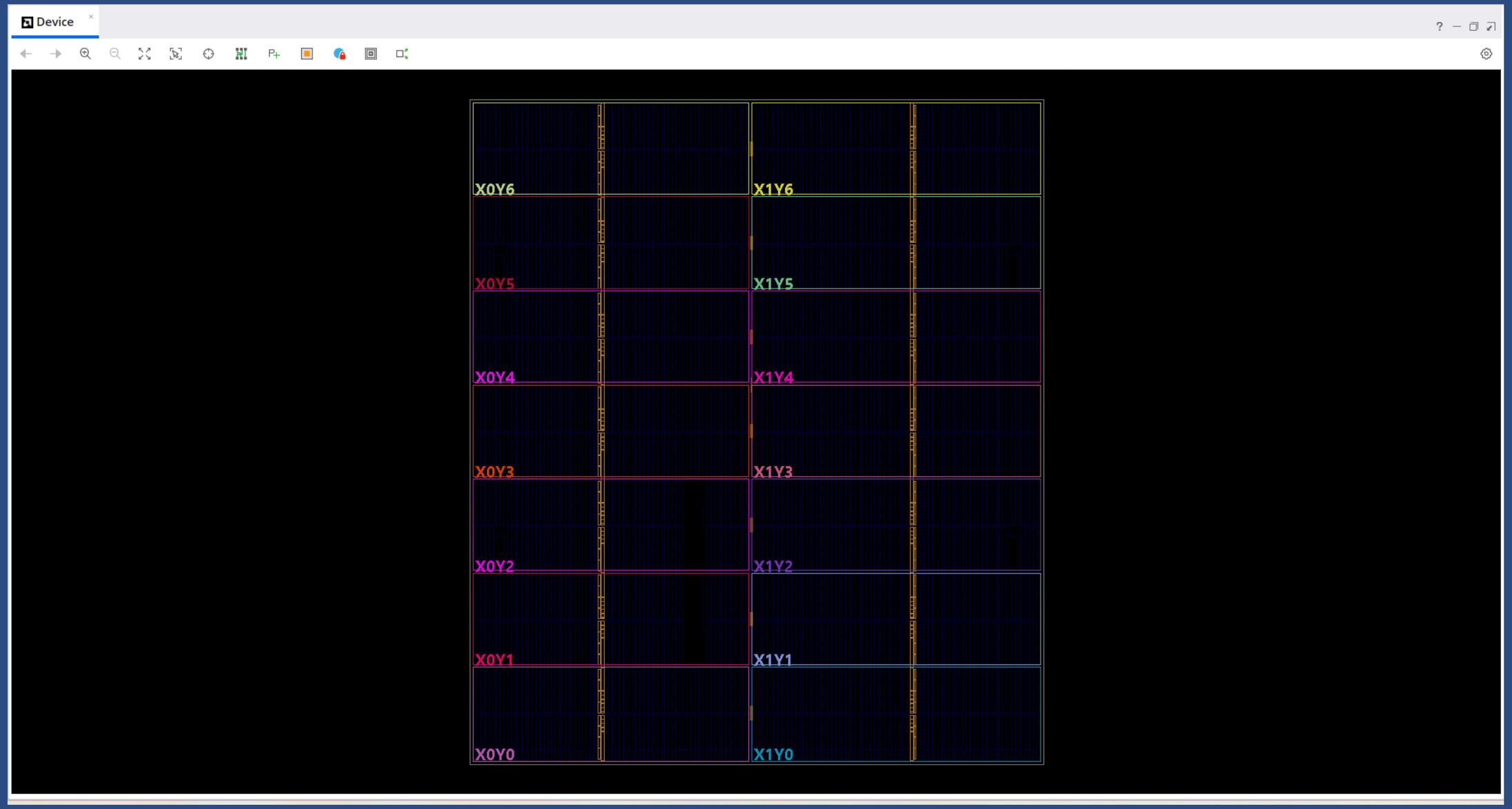
Tcl Console

```
t=            19625000 PC=a8 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19635000 PC=a9 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19645000 PC=aa | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19655000 PC=ab | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19665000 PC=ac | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19675000 PC=ad | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19685000 PC=ae | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19695000 PC=af | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19705000 PC=b0 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19715000 PC=b1 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19725000 PC=b2 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19735000 PC=b3 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19745000 PC=b4 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19755000 PC=b5 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19765000 PC=b6 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19775000 PC=b7 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19785000 PC=b8 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19795000 PC=b9 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19805000 PC=ba | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19815000 PC=bb | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19825000 PC=bc | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19835000 PC=bd | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19845000 PC=be | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19855000 PC=bf | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19865000 PC=c0 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19875000 PC=c1 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19885000 PC=c2 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19895000 PC=c3 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19905000 PC=c4 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19915000 PC=c5 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19925000 PC=c6 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19935000 PC=c7 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19945000 PC=c8 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19955000 PC=c9 | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19965000 PC=ca | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19975000 PC=cb | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19985000 PC=cc | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
t=            19995000 PC=cd | ID: 0000 | EX:v1 | MEM:v1 | WB:- R0<=00 | Z=0 C=0
TIMEOUT
$finish called at time : 20 us : File "C:/8bitCPU/8bitnguyendinhdunghe201882/8bitnguyendinhdunghe201882.srcs/sim_1/new/tb_risc8_5stage.v" Line 139
```

schematic



synthesis


Device
X0Y6
X1Y6
X0Y5
X1Y5
X0Y4
X1Y4
X0Y3
X1Y3
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0